Comparable vs Comparator

1. Comparable

- **Package**: `java.lang`
- **Interface**: `Comparable<T>`
- **Method**: `int compareTo(T o)`
- **Purpose**: Defines the *natural ordering* of objects.
- **When to use**: If you want a class to have a **default sorting logic**.
- **Limitation**: A class can only implement **one** `compareTo` method, so it can have only one natural order.

Example

```
class User implements Comparable<User> {
    private int id;
    private String name;

    @Override
    public int compareTo(User other) {
        return Integer.compare(this.id, other.id); // sort by id
    }
}

```

Usage:

```
List<User> users = ...
Collections.sort(users); // uses compareTo()

```

---

2. Comparator

- **Package**: `java.util`
- **Interface**: `Comparator<T>`
- **Method**: `int compare(T o1, T o2)`
- **Purpose**: Defines a **custom ordering** (can have multiple strategies).
- **When to use**: If you want **different sorting logics** for the same class.
- **Flexibility**: You can create as many comparators as you like.

Example

```
class UserNameComparator implements Comparator<User> {
    @Override
    public int compare(User u1, User u2) {
        return u1.getName().compareTo(u2.getName()); // sort by name
    }
}

```

Usage:

```
List<User> users = ...
Collections.sort(users, new UserNameComparator()); // uses compare()
```

```
```

---

3. Key Differences Table

Aspect	Comparable	Comparator
Package	`java.lang`	`java.util`
Method	`compareTo(T o)`	`compare(T o1, T o2)`
Ordering	Natural (default)	Custom (multiple possible)
Implementation	Implemented by the class itself	Implemented as separate class/lambda
Number allowed	Only one per class	Multiple comparators per class

---

✅ **Rule of thumb**:

- Use **Comparable** when there is **one natural order** (e.g., numbers, dates, alphabetical).
- Use **Comparator** when you need **multiple sorting strategies** (e.g., sort Users by age, then by name).